

הצגת מאמר:

CODEXGRAPH

Bridging Large Language Models and Code Repositories via Code Graph Databases (2024)

<https://arxiv.org/pdf/2408.03910>

Submitted by:

Ohad Livay, Yotam Katz

Lecturer:

Prof. Iris Reinhartz-Berger

קורס:

סמינר: הנדסת תוכנה בעידן ה-AI

01 3500 26

הבעיה



צורך בהקשר רלוונטי

מודלי שפה גדולים (LLMs) שעובדים על מאגרי קוד אמיתיים זקוקים להקשר רלוונטי מתוך בסיס הקוד לביצוע משימות כמו תיקון באגים והשלמת קוד עם תלות בין קבצים.



המודל לא מכיר את הקודבייס

RACG מבוסס דמיון: (BM25, אמבדינגים) אינו מבין את מבנה הקוד.

כלים ייעודיים: (AutoCodeRover) עובדים טוב על מדד ספציפי אך אינם מכלילים היטב.

תרומה מרכזית של המאמר

דרך חדשנית לשפר RACG: המרת מאגר הקוד למסד נתונים גרפי (כמו **Neo4j**) ונתינת האפשרות למודל להפעיל ולקבל תשובות משאילות שפועלות על הגרף ומאחזרות קוד.

מנגנון הפעולה

Primary Agent: אחראי על תרגום המשימה. כותב שאילות בשפה טבעית על בסיס משימת הקוד.

Translation Agent: מקבל מהסוכן את השאילות בשפה הטבעית ומתרגם לשאילות אחזור בגרף.

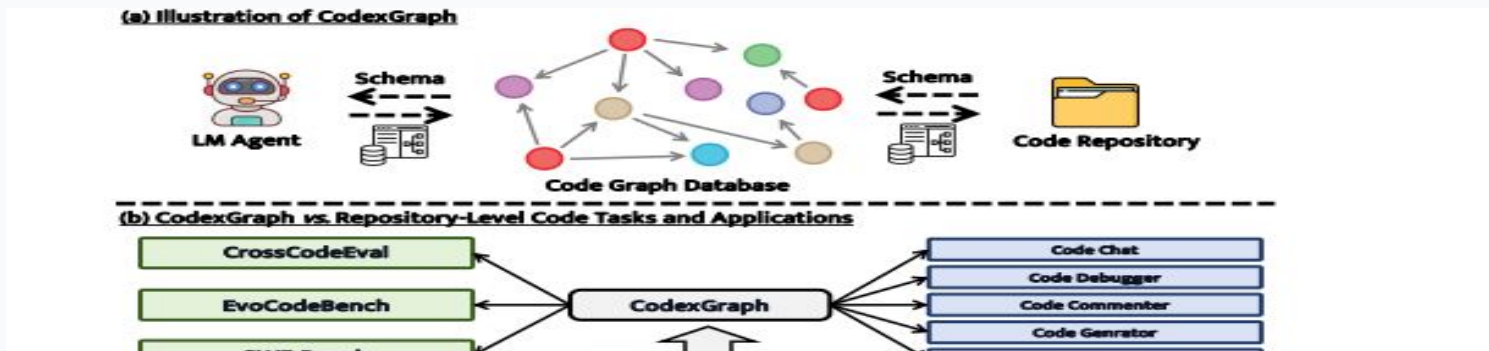
Iterative Loop: ביצוע שאילות חוזרות עד להשגת הקשר מספק.

מבנה הגרף (Graph Structure)

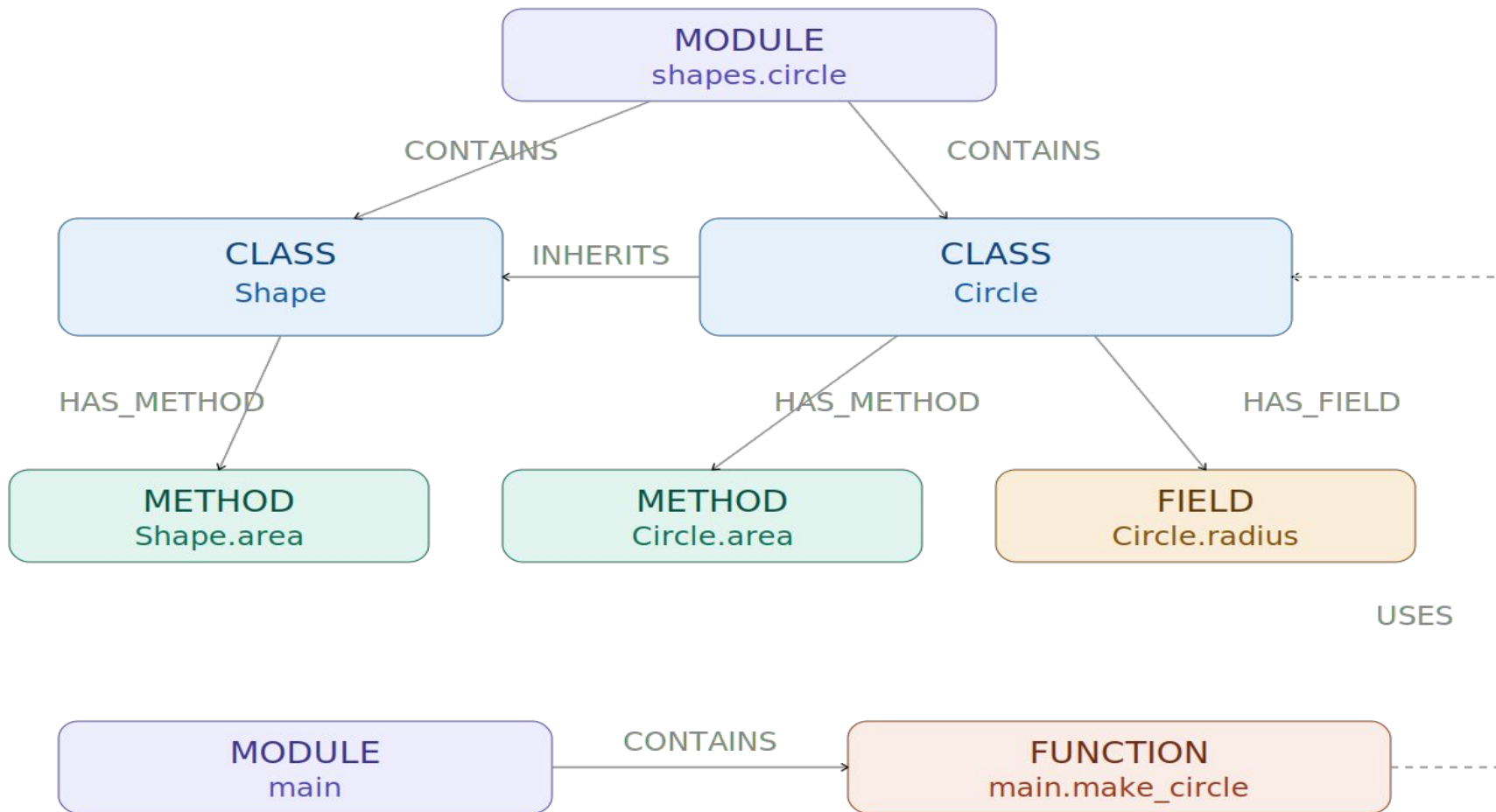
Nodes: MODULE, CLASS, METHOD, FUNCTION, FIELD, GLOBAL_VARIABLE

Edges: CONTAINS, INHERITS, HAS_METHOD, HAS_FIELD, USES

אינדוקס דו-שלבי: סריקה מהירה ומעבר DFS למילוי קשרים בין קבצים.



Graph representation





CodexGraph uses a **two-stage static analysis pipeline** to map codebases. It starts with single-file indexing (AST) and completes the graph by resolving complex cross-file dependencies using DFS.

התוצאות: ביצועים והערכה

Model	Method	CrossCodeEval Lite (Python)				SWE-bench Lite	EvoCodeBench	
		EM	ES	ID-EM	ID-F1	Pass@1	Pass@1	Recall@1
Qwen2	No RAG	8.20	46.16	13.0	36.92	-	19.34	11.34
	BM25	15.50	51.74	22.60	45.44	0.00	-	-
	AUTOCODEROVER	5.21	47.63	10.16	36.54	9.34	16.91	7.86
	CODEXGRAPH	5.00	47.99	9.10	36.44	1.95	14.62	8.60
DS-Coder	No RAG	11.70	60.73	16.90	47.85	-	25.47	11.04
	BM25	21.90	67.52	30.60	59.04	1.17	-	-
	AUTOCODEROVER	14.90	59.78	22.30	51.34	15.56	20.28	7.56
	CODEXGRAPH	20.20	63.14	28.10	54.88	12.06	27.62	12.01
GPT-4o	No RAG	10.80	59.36	16.70	48.22	-	27.83	11.79
	BM25	21.20	66.18	30.20	58.71	3.11	-	-
	AUTOCODEROVER	21.20	61.92	28.10	54.81	22.96	28.78	11.17
	CODEXGRAPH	27.90	67.98	35.60	61.08	22.96	36.02	11.87

EM: Exact match **ES:** Edit Similarity **ID-EM:** identifier EM **ID-F1:** FP,FN of correct retrieval

Ablation of Translation Agent

Table 3: Ablation study about the translation LLM agent on CrossCodeEval Lite (Python).

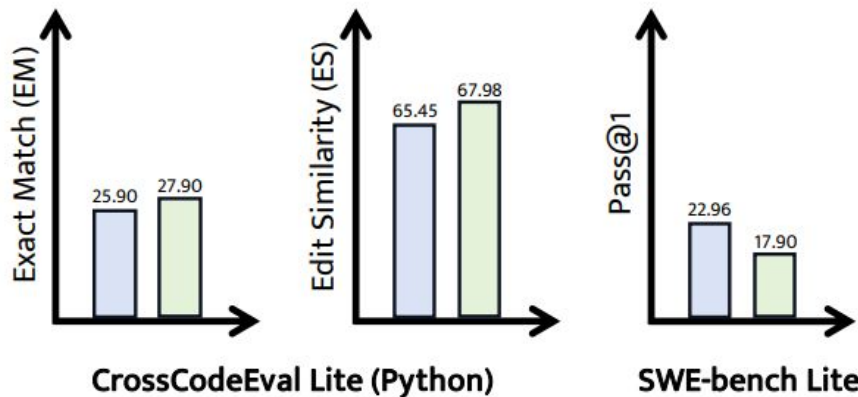
Model	Method	CrossCodeEval Lite (Python)			
		EM	ES	ID-EM	ID-F1
Qwen2	CODINGGRAPH	5.00	47.99	9.10	36.44
	w/o translation LLM Agent	0.50 (-4.50)	10.45 (-37.54)	0.60 (-8.50)	2.62 (-33.82)
DS-Coder	CODINGGRAPH	20.20	63.14	28.10	54.88
	w/o translation LLM Agent	5.50 (-14.70)	53.56 (-9.58)	11.20 (-16.90)	39.75 (-15.13)
GPT-4o	CODINGGRAPH	27.90	67.98	35.60	61.08
	w/o translation LLM Agent	8.30 (-19.60)	56.36 (-11.62)	14.40 (-21.20)	44.08 (-17.00)

פירוק משימה לגורמים
←
סוכן יכול להתמקד
←
מעלה ביצועים ללא תלות במודל!

מה הגבול?
כמה צריך\ כדאי לפרק
משימות?

Single query VS Multiple queries

Legends: ■ single query ■ multiple queries



בכל איטרציה - כמה שאילתות נייצר?
יותר שאילתות - יותר FP
פחות שאילתות - יותר FN

SWE-bench - requires harder reasoning
CrossCodeEval - requires repo-wide context

ביקורת

Remark: During indexing of 43 Sympy samples from the SWE-bench dataset, we face out-of-memory issues due to numerous files and complex dependencies, leading to their exclusion. Similarly, some EvoCodeBench samples are omitted due to test environment configuration issues. Thus, SWE-bench Lite and EvoCodeBench results are based on 257 and 212 samples, respectively.

בתיאוריה: CodexGraph נועד להתגבר על קודבייס גדול מדי בשביל להכניס לקונטקסט של המודל

אבל החוקרים בעצמם נתקלו בקשיים הקשורים בגודל ומורכבות הקודבייס.

ביקורת

חסרים הפרומפטים עבור שני הסוכנים הראשיים.

חסר פירוט איך נבנה הפרומפט הדינמי שכולל את הקוד הרלוונטי (append? קריאה חדשה? המשך ה-thread?)

אין הסבר מדוע נבחרה טמפרטורה 0 עבור gpt 4o/qwen2 אך טמפרטורה 1 עבור deepseek

נבדק רק על פייתון וכדי לבדוק יכולת הכללה של מערכת כדאי לבדוק שפות שונות מהותית

שיפורים, הצעות ותהיות

1. הרחבת ה-usecase לסוגים נוספים של גרפים:
 - a. דטאבייס רלציוני בשביל משימות Exploratory Data Analysis
 - b. מאגר מידע ארגוני לצורך RAG
 - c. ...
2. לחקור השפעת היפרפרמטרים של המודל כמו טמפרטורה\prompt\effort\TOP-K על ביצועים
3. לחקור השפעת כמות הטוקנים וקריאות ל-LLM המותרות על ביצועי המודל
4. מעניין לדעת אם פירוק משימה לגורמים עדיין רלוונטי כיום עבור מודלים עם context window גדול (למודל Gpt 4o היה 128K, כיום opus4.6/gemini 3.1pro עם יותר ממיליון!)

הפעלה

אנחנו, מחלקת R&D, רוצים לפתח מערכת תומכת SWE בהשראת CodexGraph (אבל לא בהכרח 1:1) ביחד אנחנו צריכים להחליט על הארכיטקטורה. על כל שאלה ינתנו מספר אופציות כדוגמה - אין פתרון אחד נכון. אפשר להציע פתרון חדש, לשלב בין האופציות, לדון ביתרונות ובחסרונות של כל גישה.

רצוי מאוד לקחת בחשבון היבטי פיתוח תוכנה שונים: זמן פיתוח, כמות טוקנים לשימוש ממוצע, זמני ריצה, מורכבות הפרויקט וסיכונים, היבטי אבטחת מידע, איכות וכו'.

הפעלה - שאלה 1

באיזה שלב של ריצת המערכת הגרף יבנה?

1. מיד עם קבלת הקודבייס (Pre-process)
2. בזמן שימוש (

הפעלה - שאלה 2

איך כדאי להנגיש את הגרף למודל?

1. בעזרת graph query language כלשהי

2. בעזרת פונקציות מוגדרות מראש כמו `get_methods(class)`, `get_children(object)`

הפעלה - שאלה 3

איך כדאי לבנות את הגרף של הקוד?

1. מודל LLM דמוי Crawler (יבצע ls, grep)
2. קוד שיכול לנתח את הסינטקס של השפה ולייצר את הגרף באופן דטרמיניסטי

הפעלה - שאלה 4

איך כדאי שיראו ה-nodes וה-vertices של הגרף?

1. רזולוציה גבוהה: כל מתודה, מחלקה, משתנה - node.
2. רזולוציה נמוכה: רמת הקבצים.

Bonus round

אילו שיקולים חשוב לקחת בחשבון כאשר תבחרו בין שימוש במודל Open-source לוקאלי (qwen,gemma) לבין מודל enterprise (opus,gpt,gemini) כמו

איך תדעו להחליט האם מודל A יותר מתאים למשימה ממודל B?

איך תמדדו האם שימוש במערכת שבנינו אכן עוזר למתכנתים שלנו?

בעיית הקונטקסט - רלוונטיות מתמיד!

